```
# ============================================
# 1. CONFIGURACIÓN DEL ENTORNO (GOOGLE COLAB)
# ============================================

!pip install -q streamlit pyngrok pandas numpy scikit-learn plotly
!pip install -q streamlit-plotly-events
# Permite ejecutar Streamlit dentro de Colab
from pyngrok import ngrok
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 78.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 95.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.8/14.8 MB 76.4 MB/s eta 0:00:00
```

```
# ============================================
# 2. IMPORTACIÓN DE LIBRERÍAS
# ============================================

import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

```
# ============================================
# 3. CARGA DINÁMICA DEL DATASET
# ============================================

def load_dataset(uploaded_file):
    """
    Carga el dataset de forma dinámica.
    Permite reutilizar el sistema con diferentes fuentes de datos.
    """
    if uploaded_file.name.endswith('.csv'):
        return pd.read_csv(uploaded_file)
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
        st.error("Formato no soportado")
        return None
```

```
# ============================================
# 4. CÁLCULO DE VARIABLES RFM
# ============================================

def calculate_rfm(df, customer_col, date_col, amount_col):
    df[date_col] = pd.to_datetime(df[date_col])

    # Limpieza estricta: Forzar numérico para evitar colapso de sum() en variables datetime/string
    df[amount_col] = pd.to_numeric(df[amount_col], errors='coerce')
    df = df.dropna(subset=[amount_col])

    reference_date = df[date_col].max() + pd.Timedelta(days=1)

    rfm = df.groupby(customer_col).agg({
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })

    rfm.columns = ['Recency', 'Frequency', 'Monetary']
    return rfm.reset_index()
```

```
# ============================================
# 5. NORMALIZACIÓN Y MÉTRICAS DE VALIDACIÓN
# ============================================

def scale_features(rfm_df):
    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_df[['Recency','Frequency','Monetary']])
    return scaled

def calculate_elbow_silhouette(data, max_k=10):
    inertia = []
    silhouette = []
    for k in range(2, max_k+1):
        model = KMeans(n_clusters=k, random_state=42)
```

```
        labels = model.fit_predict(data)
        inertia.append(model.inertia_)
        silhouette.append(silhouette_score(data, labels))
    return inertia, silhouette
```

```
# ============================================
# 6. ENTRENAMIENTO DEL MODELO K-MEANS (K)
# ============================================

def train_kmeans(data, k):
    """Entrena el modelo K-Means usando el K óptimo descubierto."""
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels
```

```
# ============================================
# 7. ASIGNACIÓN DE SEGMENTOS RFM (AJUSTADA)
# ============================================

def assign_dynamic_segments(rfm):
    """
    Mapeo dinámico: Evalúa matemáticamente los centroides descubiertos.
    Score = Monetary_z + Frequency_z - Recency_z.
    """
    centroids = rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean()

    scaler = StandardScaler()
    scaled_centroids = pd.DataFrame(
        scaler.fit_transform(centroids),
        columns=['R_z', 'F_z', 'M_z'],
        index=centroids.index
            )

    scaled_centroids['Score'] = scaled_centroids['M_z'] + scaled_centroids['F_z'] - scaled_centroids['R_z']
    scaled_centroids = scaled_centroids.sort_values('Score', ascending=False)

    cluster_to_label = {}
    for idx, (cluster_id, row) in enumerate(scaled_centroids.iterrows()):
        if idx == 0:
            cluster_to_label[cluster_id] = 'VIP'
        elif idx == len(scaled_centroids) - 1:
            cluster_to_label[cluster_id] = 'Perdidos'
        else:
            if row['R_z'] > 0:
                cluster_to_label[cluster_id] = 'En Riesgo'
            else:
                cluster_to_label[cluster_id] = 'Leales'

    rfm['Segmento'] = rfm['Cluster'].map(cluster_to_label)
    return rfm
```

```
# ============================================
# 8. RECOMENDACIONES ESTRATÉGICAS DE MARKETING
# ============================================

def marketing_recommendations(segment):
    recommendations = {
        "VIP": [
            "Ofrecer programas de fidelización exclusivos",
            "Acceso anticipado a promociones y lanzamientos",
            "Comunicación personalizada 1 a 1",
            "Incentivos por referidos"
        ],
        "Leales": [
            "Descuentos progresivos por frecuencia de compra",
            "Campañas de cross-selling y up-selling",
            "Programas de puntos",
            "Encuestas de satisfacción"
        ],
        "En Riesgo": [
            "Campañas de reactivación con descuentos agresivos",
            "Recordatorios personalizados por email/SMS",
            "Ofertas por tiempo limitado",
            "Análisis de causas de abandono"
        ],
        "Perdidos": [
            "Campañas de recuperación de bajo costo",
            "Ofertas de bienvenida para reenganche",
            "Excluir de campañas costosas",
```

```
            "Evaluar eliminación del CRM activo"
        ]
    }

    return recommendations.get(segment, [])
```

```
# ==========================================
# 9. DASHBOARD INTERACTIVO STREAMLIT
# ==========================================

st.title("Segmentación de Clientes – Retail (RFM + K-Means)")

uploaded_file = st.file_uploader("Cargar dataset transaccional")

if uploaded_file:
    df = load_dataset(uploaded_file)

    col1, col2, col3 = st.columns(3)
    with col1: customer_col = st.selectbox("Columna ID Cliente", df.columns)
    with col2: date_col = st.selectbox("Columna Fecha", df.columns)
    with col3: amount_col = st.selectbox("Columna Monto", df.columns)

    rfm = calculate_rfm(df, customer_col, date_col, amount_col)
    scaled = scale_features(rfm)

    st.markdown("---")
    st.subheader("Optimización del Número de Clusters ($K$)")

    inertia, silhouette = calculate_elbow_silhouette(scaled, max_k=10)
    best_k = int(np.argmax(silhouette) + 2)

    sil_df = pd.DataFrame({'K': range(2, 11), 'Silhouette Score': silhouette})

    col_plot, col_slider = st.columns([2, 1])
    with col_plot:
        st.line_chart(sil_df.set_index('K'))
    with col_slider:
        st.write(f"**$K$ Óptimo matemático:** {best_k}")
        k_elegido = st.slider("Selecciona $K$", min_value=2, max_value=10, value=best_k)

    # Entrenar modelo con K dinámico
    model, labels = train_kmeans(scaled, k_elegido)
    rfm['Cluster'] = labels

    # Asignar etiquetas dinámicamente
    rfm = assign_dynamic_segments(rfm)

    st.markdown("---")
    col_bar, col_3d = st.columns([1, 2])

    with col_bar:
        st.subheader("Distribución de Segmentos")
        st.bar_chart(rfm['Segmento'].value_counts())

    with col_3d:
        st.subheader("Topología 3D")
        fig = px.scatter_3d(
            rfm, x='Recency', y='Frequency', z='Monetary',
            color='Segmento', opacity=0.8,
            title="Distribución RFM por Segmento"
        )
        fig.update_layout(margin=dict(l=0, r=0, b=0, t=30))
        st.plotly_chart(fig, use_container_width=True)

    st.markdown("---")
    st.subheader("Recomendaciones de Marketing")
    segmentos_disponibles = rfm['Segmento'].unique().tolist()
    selected_segment = st.selectbox("Selecciona un segmento descubierto", segmentos_disponibles)

    recs = marketing_recommendations(selected_segment)
    for r in recs:
        st.markdown(f"✅ {r}")
```

```
2026-02-19 21:00:50.696 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.413
  Warning: to view this Streamlit app on a browser, run it with the following
  command:

    streamlit run /usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py [ARGUMENTS]
2026-02-19 21:00:51.416 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.419 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.422 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.426 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
2026-02-19 21:00:51.428 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.433 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.438 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 21:00:51.441 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
# Ejecutar Streamlit en Colab
# Obtén tu authtoken de ngrok en https://dashboard.ngrok.com/get-started/your-authtoken
# y reemplaza 'YOUR_AUTHTOKEN' con tu token real.
ngrok.set_auth_token('38CulDXlM53o6OGkHScEziK918U_4TJ3Erf2M63j3FoxhJqPX')
public_url = ngrok.connect(8501)
print(public_url)

# Guardar el código de Streamlit en un archivo app.py
with open('app.py', 'w') as f:
    f.write("""
import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# ============================================
# 3. CARGA DINÁMICA DEL DATASET
# ============================================
def load_dataset(uploaded_file):
    \"\"\"
    Carga el dataset de forma dinámica.
    Permite reutilizar el sistema con diferentes fuentes de datos.
    \"\"\"
    if uploaded_file.name.endswith('.csv'):
        return pd.read_csv(uploaded_file)
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
        st.error("Formato no soportado")
        return None

# ============================================
# 4. CÁLCULO DE VARIABLES RFM
# ============================================
def calculate_rfm(df, customer_col, date_col, amount_col):
    df[date_col] = pd.to_datetime(df[date_col])

    # Limpieza estricta: Forzar numérico para evitar colapso de sum() en variables datetime/string
    df[amount_col] = pd.to_numeric(df[amount_col], errors='coerce')
    df = df.dropna(subset=[amount_col])

    reference_date = df[date_col].max() + pd.Timedelta(days=1)

    rfm = df.groupby(customer_col).agg({
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })

    rfm.columns = ['Recency', 'Frequency', 'Monetary']
    return rfm.reset_index()

# ============================================
# 5. NORMALIZACIÓN Y MÉTRICAS DE VALIDACIÓN
# ============================================
def scale_features(rfm_df):
    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_df[['Recency','Frequency','Monetary']])
    return scaled

def calculate_elbow_silhouette(data, max_k=10):
    inertia = []
    silhouette = []
    for k in range(2, max_k+1):
        model = KMeans(n_clusters=k, random_state=42)
        labels = model.fit_predict(data)
        inertia.append(model.inertia_)
        silhouette.append(silhouette_score(data, labels))
    return inertia, silhouette

# ============================================
# 6. ENTRENAMIENTO DEL MODELO K-MEANS
# ============================================
def train_kmeans(data, k):
    \"\"\"Entrena el modelo K-Means usando el K óptimo descubierto.\"\"\"
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels
```

```
# ==========================================
# 7. ASIGNACIÓN DINÁMICA DE SEGMENTOS RFM
# ==========================================
def assign_dynamic_segments(rfm):
    \"\"\"
    Mapeo dinámico: Evalúa matemáticamente los centroides descubiertos.
    Score = Monetary_z + Frequency_z - Recency_z.
    \"\"\"
    centroids = rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean()

    scaler = StandardScaler()
    scaled_centroids = pd.DataFrame(
        scaler.fit_transform(centroids),
        columns=['R_z', 'F_z', 'M_z'],
        index=centroids.index
    )

    scaled_centroids['Score'] = scaled_centroids['M_z'] + scaled_centroids['F_z'] - scaled_centroids['R_z']
    scaled_centroids = scaled_centroids.sort_values('Score', ascending=False)

    cluster_to_label = {}
    for idx, (cluster_id, row) in enumerate(scaled_centroids.iterrows()):
        if idx == 0:
            cluster_to_label[cluster_id] = 'VIP'
        elif idx == len(scaled_centroids) - 1:
            cluster_to_label[cluster_id] = 'Perdidos'
        else:
            if row['R_z'] > 0:
                cluster_to_label[cluster_id] = 'En Riesgo'
            else:
                cluster_to_label[cluster_id] = 'Leales'

    rfm['Segmento'] = rfm['Cluster'].map(cluster_to_label)
    return rfm

# ==========================================
# 8. RECOMENDACIONES ESTRATÉGICAS DE MARKETING
# ==========================================
def marketing_recommendations(segment):
    recommendations = {
        "VIP": [
            "Programas de fidelización premium",
            "Ofertas exclusivas y lanzamientos anticipados",
            "Beneficios por referidos",
            "Atención personalizada"
        ],
        "Leales": [
            "Promociones por frecuencia",
            "Cross-selling y up-selling",
            "Programa de puntos",
            "Comunicación periódica personalizada"
        ],
        "En Riesgo": [
            "Campañas de reactivación",
            "Descuentos por tiempo limitado",
            "Recordatorios personalizados",
            "Análisis de churn"
        ],
        "Perdidos": [
            "Campañas de recuperación de bajo costo",
            "Ofertas de reenganche",
            "Excluir de campañas costosas",
            "Depuración del CRM"
        ]
    }
    return recommendations.get(segment, [])

# ==========================================
# 9. DASHBOARD INTERACTIVO STREAMLIT
# ==========================================
st.title("Segmentación de Clientes – Retail (RFM + K-Means)")

uploaded_file = st.file_uploader("Cargar dataset transaccional")

if uploaded_file:
    df = load_dataset(uploaded_file)

    col1, col2, col3 = st.columns(3)
    with col1: customer_col = st.selectbox("Columna ID Cliente", df.columns)
    with col2: date_col = st.selectbox("Columna Fecha", df.columns)
    with col3: amount_col = st.selectbox("Columna Monto", df.columns)

    rfm = calculate_rfm(df, customer_col, date_col, amount_col)
    scaled = scale_features(rfm)

    st.markdown("---")
    st.subheader("Optimización del Número de Clusters ($K$)")
```

```
    inertia, silhouette = calculate_elbow_silhouette(scaled, max_k=10)
    best_k = int(np.argmax(silhouette) + 2)

    sil_df = pd.DataFrame({'K': range(2, 11), 'Silhouette Score': silhouette})

    col_plot, col_slider = st.columns([2, 1])
    with col_plot:
        st.line_chart(sil_df.set_index('K'))
    with col_slider:
        st.write(f"**$K$ Óptimo matemático:** {best_k}")
        k_elegido = st.slider("Selecciona $K$", min_value=2, max_value=10, value=best_k)

    # Entrenar modelo con K dinámico
    model, labels = train_kmeans(scaled, k_elegido)
    rfm['Cluster'] = labels

    # Asignar etiquetas dinámicamente
    rfm = assign_dynamic_segments(rfm)

    st.markdown("---")
    col_bar, col_3d = st.columns([1, 2])

    with col_bar:
        st.subheader("Distribución de Segmentos")
        st.bar_chart(rfm['Segmento'].value_counts())

    with col_3d:
        st.subheader("Topología 3D")
        fig = px.scatter_3d(
            rfm, x='Recency', y='Frequency', z='Monetary',
            color='Segmento', opacity=0.8,
            title="Distribución RFM por Segmento"
        )
        fig.update_layout(margin=dict(l=0, r=0, b=0, t=30))
        st.plotly_chart(fig, use_container_width=True)

    st.markdown("---")
    st.subheader("Recomendaciones de Marketing")
    segmentos_disponibles = rfm['Segmento'].unique().tolist()
    selected_segment = st.selectbox("Selecciona un segmento descubierto", segmentos_disponibles)

    recs = marketing_recommendations(selected_segment)
    for r in recs:
        st.markdown(f"✅ {r}")
""")

# Ahora, ejecuta el archivo app.py con streamlit
!streamlit run app.py
```

```
NgrokTunnel: "https://overcoolly-cycadaceous-colten.ngrok-free.dev" -> "http://localhost:8501"

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.28.0.12:8501
  External URL: http://34.125.130.242:8501

────────────────────────── Traceback (most recent call last) ──────────────────────────
  /usr/local/lib/python3.12/dist-packages/streamlit/runtime/scriptrunner/exec_code.py:
  129 in exec_func_with_error_handling

  /usr/local/lib/python3.12/dist-packages/streamlit/runtime/scriptrunner/script_runner
  .py:689 in code_to_exec

  /content/app.py:157 in <module>

    154 │   with col2: date_col = st.selectbox("Columna Fecha", df.columns)
    155 │   with col3: amount_col = st.selectbox("Columna Monto", df.columns)
    156 │
  ❱ 157 │   rfm = calculate_rfm(df, customer_col, date_col, amount_col)
    158 │   scaled = scale_features(rfm)
    159 │
    160 │   st.markdown("---")

  /content/app.py:37 in calculate_rfm

     34 │   df[amount_col] = pd.to_numeric(df[amount_col], errors='coerce')
     35 │   df = df.dropna(subset=[amount_col])
     36 │
  ❱  37 │   reference_date = df[date_col].max() + pd.Timedelta(days=1)
     38 │
     39 │   rfm = df.groupby(customer_col).agg({
     40 │   │   date_col: lambda x: (reference_date - x.max()).days,
────────────────────────────────────────────────────────────────────────────────────────
TypeError: unsupported operand type(s) for +: 'int' and 'Timedelta'
────────────────────────── Traceback (most recent call last) ──────────────────────────
```

```
/usr/local/lib/python3.12/dist-packages/streamlit/runtime/scriptrunner/exec_code.py:
129 in exec_func_with_error_handling

/usr/local/lib/python3.12/dist-packages/streamlit/runtime/scriptrunner/script_runner
.py:689 in code_to_exec

/content/app.py:157 in <module>

  154 |   with col2: date_col = st.selectbox("Columna Fecha", df.columns)
  155 |   with col3: amount_col = st.selectbox("Columna Monto", df.columns)
  156 |
❯ 157 |   rfm = calculate_rfm(df, customer_col, date_col, amount_col)
  158 |   scaled = scale_features(rfm)
  159 |
  160 |   st.markdown("---")

/content/app.py:37 in calculate_rfm
```